

1. [2p] Dado el siguiente código Java,

- Dibujar un diagrama de clases UML que muestre todas las clases e interfaces involucradas y sus relaciones.
- Dibujar un diagrama de secuencia UML que muestre la interacción entre los objetos durante la ejecución de la función main.
- ¿Cómo se manifiesta el polimorfismo en este código? Justificar.

Nota: Incluir la clase anónima en ambos diagramas, marcada como <<anonima>>.

Notificador.java

```
1 public interface Notificador {
2     void enviar(String msj);
3 }
```

NotificadorEmail.java

```
1 public class NotificadorEmail implements Notificador {
2     public void enviar(String msj) {
3         System.out.println("Email: " + msj);
4     }
5 }
```

Usuario.java

```
1 public class Usuario {
2     private List<Notificador> notificadores = new
        ArrayList<>();
3
4     public void agregarNotificador(Notificador n) {
5         notificadores.add(n);
6     }
7
8     public void notificar(String msj) {
9         for (Notificador n : notificadores) {
10             n.enviar(msj);
11         }
12     }
13 }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         Usuario u = new Usuario();
4         u.agregarNotificador(new NotificadorEmail());
5         u.agregarNotificador(new Notificador() {
6             public void enviar(String msj) {
7                 System.out.println("Log: " + msj);
8             }
9         });
10        u.notificar("Alerta");
11    }
12 }
```

Crterios de corrección:

• Diagrama de clases:

- Usuario tiene una relación de agregación/composición con Notificador.
- Notificador es interfaz; NotificadorEmail la implementa.
- La clase anónima en Main (marcada con <<anonima>>) también implementa Notificador.
- Main depende de Usuario y NotificadorEmail.

• Diagrama de secuencia:

- Hay 4 objetos: :Main, u:Usuario, n1:NotificadorEmail, n2:<<anonima>>.
- :Main instancia u:Usuario
- :Main instancia n1:NotificadorEmail
- :Main invoca u.agregarNotificador(n1)
- :Main instancia la clase anónima y la agrega también.
- :Main invoca u.notificar("Alerta")
- u:Usuario recorre su lista y envía el mensaje enviar("Alerta") a cada instancia de Notificador.

• Polimorfismo:

- La línea n.enviar(msj) en Usuario.java.
- Justificación: n es de tipo estático Notificador, pero el método que se ejecuta depende del tipo dinámico del objeto (ej: NotificadorEmail o la clase anónima).

2. [1p] Responda basándose en el código del ejercicio anterior:

- a) ¿El diseño de la clase Usuario respeta el principio de Abierto/Cerrado (OCP) ante la necesidad de agregar nuevos medios de notificación (por ejemplo, SMS)? Justificar.
- b) ¿Qué principio de diseño se viola si agregamos UsuarioPremium al sistema? Justificar.

```
1 public class UsuarioPremium extends Usuario {
2     @Override
3     public void notificar(String msj) {
4         if (super.notificadores.isEmpty()) {
5             throw new IllegalStateException("No hay notificadores configurados");
6         }
7         super.notificar(msj);
8     }
9 }
```

Criterios de corrección:

- **OCP:** Sí respeta. La clase Usuario no requiere ser modificada para soportar nuevos medios de notificación; basta con implementar la interfaz Notificador.
- **LSP:** Principio de Sustitución de Liskov. Al agregar una pre-condición más fuerte (que la lista no esté vacía) o cambiar el comportamiento esperado lanzando una excepción que el cliente no espera manejar según el contrato de Usuario, se rompe la posibilidad de usar UsuarioPremium en lugar de Usuario.

3. [2p] Dado el siguiente programa Prolog que describe vuelos directos y rutas entre ciudades, mostrar el seguimiento de la consulta `?- ruta(bsas, paris, T).`, indicando todos los valores de T en el orden en que Prolog los encontraría.

```
1 vuelo(bsas, madrid, 12).
2 vuelo(madrid, paris, 2).
3 vuelo(madrid, roma, 2).
4 vuelo(roma, paris, 1).
5
6 ruta(X, Y, T) :- vuelo(X, Y, T).
7 ruta(X, Y, T) :-
8     vuelo(X, Z, T1),
9     ruta(Z, Y, T2),
10    T is T1 + T2.
```

Criterios de corrección:

Árbol de búsqueda para `ruta(bsas, paris, T)`:

```
ruta(bsas, paris, T)
├ vuelo(bsas, madrid, 12) -> T = 12
│   └ ruta(madrid, paris, T2)
│       ├── vuelo(madrid, paris, 2) -> T2 = 2
│       │   └ T is 12 + 2 -> T = 14
│       └ vuelo(madrid, roma, 2) -> T2 = 2
│           └ ruta(roma, paris, T3)
│               └ vuelo(roma, paris, 1) -> T3 = 1
│                   └ T is 12 + 2 + 1 -> T = 15
```

Resultados: T = 14 (vía Madrid -> Paris) y T = 15 (vía Madrid -> Roma -> Paris).

4. [2p] Dado el siguiente código Clojure:

```
1 (defn transformar [n xs]
2   (map (fn [x] (* x 2))
3        (filter (fn [x] (> x n)) xs)))
4
5 (defn calcular [n xs]
6   (reduce + (transformar n xs)))
```

- a) ¿Cuál es el resultado de ejecutar (calcular 5 [2 8 4 10])? Mostrar el resultado intermedio de transformar.
- b) Suponga que xs es una secuencia infinita (ej: (range)).
- ¿La llamada (take 5 (transformar 10 (range))) termina? Justificar.
 - ¿La llamada (calcular 10 (range)) termina? Justificar.

Criterios de corrección:

- **Seguimiento:** transformar devuelve (16 20). calcular devuelve 36.
- **Lazyness:**
 - take 5 termina: transformar genera una secuencia perezosa y take solo pide los primeros 5 elementos que cumplan la condición.
 - calcular no termina: reduce intenta procesar toda la secuencia infinita para sumarla.

5. [2p] Dadas las siguientes definiciones en Cálculo Lambda:

- $\text{TRUE} = \lambda x. \lambda y. x$
- $\text{FALSE} = \lambda x. \lambda y. y$
- $\text{NOT} = \lambda p. p \text{ FALSE TRUE}$

Realizar el seguimiento paso a paso de la reducción de la siguiente expresión hasta llegar a su forma normal:

NOT FALSE

Indicar en cada paso qué reducción se está aplicando.

Criterios de corrección:

• **Reducción:**

1. $(\lambda p. p (\lambda x. \lambda y. y) (\lambda x. \lambda y. x)) (\lambda x. \lambda y. y)$ (Sustitución de definiciones)
 2. $(\lambda x. \lambda y. y) (\lambda x. \lambda y. y) (\lambda x. \lambda y. x)$ (Beta-reducción de p)
 3. $(\lambda y. y) (\lambda x. \lambda y. x)$ (Beta-reducción del primer x de FALSE)
 4. $\lambda x. \lambda y. x$ (Beta-reducción de y)
- El resultado coincide con la definición de TRUE .

6. [1p] Dado el siguiente fragmento JavaFX:

Controller.java

```
1 public class Controller {
2     @FXML
3     private Button btnProcesar;
4
5     @FXML
6     void handleProcesar(ActionEvent event) {
7         btnProcesar.setText("Procesando...");
8         try {
9             // Simulamos una tarea pesada
10            Thread.sleep(5000);
11        } catch (InterruptedException e) {
12            e.printStackTrace();
13        }
14        btnProcesar.setText("¡Listo!");
15    }
16 }
```

- a) Cuando el usuario hace click en el botón, ¿qué sucede con la ventana de la aplicación durante esos los próximos 5 segundos? Justificar usando términos y conceptos de Programación de Interfaces Gráficas y Programación Concurrente.
- b) En base al análisis del punto anterior, ¿identifica algún problema en la implementación del método `handleProcesar`? ¿Cómo debería reestructurarse el método para solucionarlo? Describir la estrategia.

Criterios de corrección:

- **Comportamiento:** La UI no cambia inmediatamente. Se congela (no responde) durante los 5 segundos. Justificación: El método se ejecuta en el **JavaFX Application Thread**; mientras este hilo esté bloqueado por `Thread.sleep`, no puede procesar eventos de repintado ni de usuario.
- **Corrección:** La tarea pesada debe ejecutarse en un hilo secundario (usando `Thread`, `Task` o `Service`). Para actualizar el texto del botón al finalizar, se debe volver al hilo de la UI usando `Platform.runLater`.